Group 8

TEE Interface Attack Simulation Project

Sebastian Alfonso

Daniel Martínez Teruel

Alejandro Montellano

Cybersecurity for embedded systems

2025

Content

1. Trusted application and host application design.
2. Vulnerabilities.
3. Attacks.
4. Demo application

Host Application Design.

- Encrypt: ta_secret –e
    1. password_validation(pwd)
    2. updateLog("Encrypting")
    3. encrypt_file(file, key)

- Decrypt: ta_secret -d
    1. password_validation(pwd)
    2. updateLog("Decrypting")
    3. decrypt_file(file, key)

- Access log: ta_secret -a
    1. password_validation(pwd)
    2. updateLog("Decrypting")
    3. getLog(file, key)

Trusted Application Design.

- Light cryptography.
  - alloc_resources(session, param_types, params)
  - set_aes_key(session, param_types, params)
  - reset_aes_iv(session, param_types, params)
  - cipher_buffer(session, param_types, params)
- Password validation.
  - password_validation(param_types, params)
- Secret management.
  - updateLog(param_types, params)
  - get_log_entry(param_types, params)

Vulnerabilities

TEE_Result updateLog()

This function is responsible for updating a log that contains only 10 entries and is stored in a persistent object. The function does not perform any synchronization to handle the object in case of multiple instantiations of the same TA, which can lead to race conditions.

```
uint32_t flags =    TEE_DATA_FLAG_ACCESS_READ |
                    TEE_DATA_FLAG_ACCESS_WRITE |
                    TEE_DATA_FLAG_ACCESS_WRITE_META |
                    TEE_DATA_FLAG_SHARE_READ |
                    TEE_DATA_FLAG_SHARE_WRITE;

res = TEE_OpenPersistentObject(TEE_STORAGE_PRIVATE,OBJ_ID, OBJ_ID_LEN,flags, &obj);
```

Vulnerabilities

TEE_Result get_log_entry()

This function returns a single row from the log; its input is the index into the log array. The main vulnerability lies in the input validation of the index, which should be limited to values between 0 and 9 but is only validated to be <9.

```
int32_t entry_index = params[1].value.a;
```

Vulnerabilities

TEE_Result password_validation()

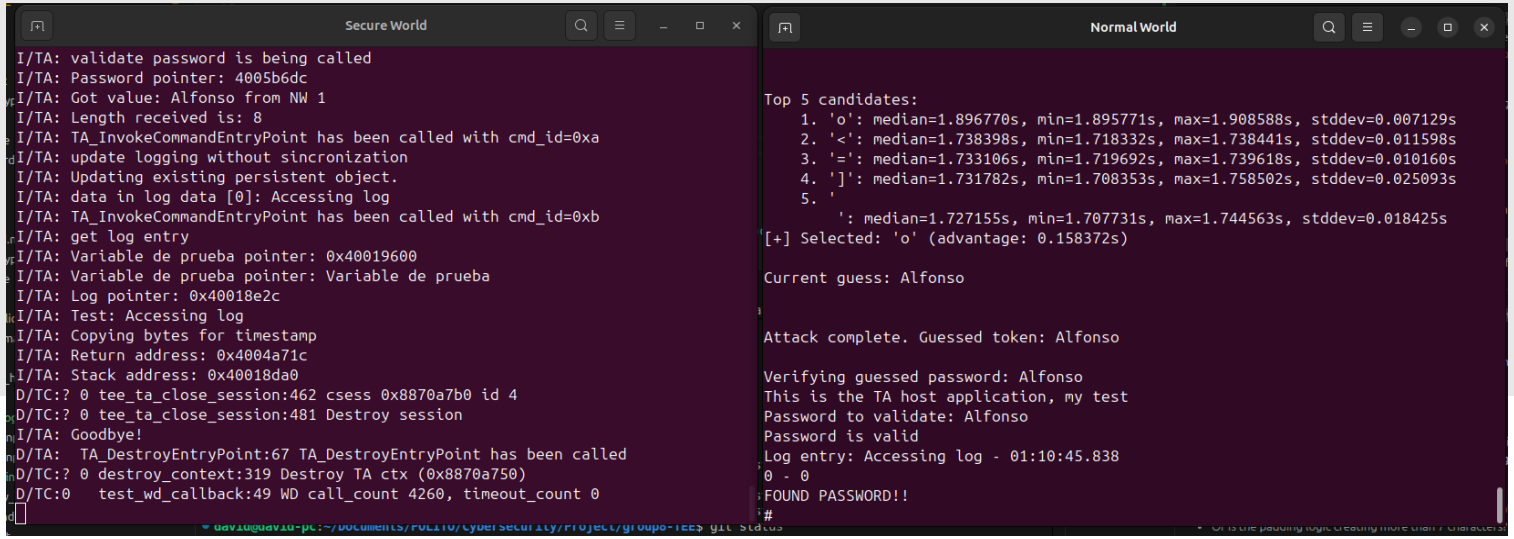
- This function simply validates the password, which is stored in a static variable.
- String validation uses strcmp, which compares characters sequentially and returns when a mismatch or the end of the string is reached.

Attacks – Timing

- The file timing.py:
  - It tests with every possible character in each position of the password
  - Based on the character that takes the longer it chooses the best fit for the position until it assembles a complete string.
- Results:
  - Timing attack was achieved in a didactical manner. It starts with a known token size as done in Lab 3.

Remarks:

- A delay had to be introduced in pass_validation.c



Attacks – Fuzzing Attack

- The file fuzzingAttack.py:
  This attack explores the effect of different inputs on the application.

  This includes:
  1. Sending fewer or more parameters than expected
  2. Using special characters in the password and encryption key
  3. Testing negative indices when accessing logs
  4. Providing extremely large numerical values
  5. The goal is to identify improper input handling that could lead to unexpected behavior or security vulnerabilities.
- Results:
  Some inputs generate segmentation fault.

Attacks - Memory Mapping

- The file memory_mapping.py:
  - It runs ta_secret -a with negative indices until it receives back an error. The received result is logged to the file memory.txt
- Results:
  - The password was retrieved on index –1328
  - Other information like errors, code or similar – indices –1506:-1546, -1550:-1584 for example

Attacks - Memory Mapping

```
Index: 1550
Log entry: %%Failed to initialize memory pool%Too large bigint%Modulus is too short%too small modulus or trying
Index: 1551
%0
Index: 1552
Log entry: %to invert zero%too small modulus%get_mpi%bigint_binary%mpi_egcd%_TEE_MathAPI_Init%TEE_BigIntInit%TEE
Index: 1553
%0
Index: 1554
Log entry: %_BigIntConvertToOctetString%TEE_BigIntConvertFromS32%bigint_binary_mod%TEE_BigIntShiftRight%TEE_BigI
Index: 1555
%0
Index: 1556
Log entry: %ntNeg%TEE_BigIntDiv%TEE_BigIntMod%TEE_BigIntInvMod%TEE_BigIntRelativePrime%TEE_BigIntExpMod%TEE_BigI
Index: 1557
%0
Index: 1558
Log entry: %ntComputeExtendedGcd%TEE_BigIntConvertFromFMM%TEE_BigIntComputeFMM%true%false%%u:%pUl%gpd.tee.arith.
Index: 1559
%0
Index: 1560
Log entry: %maxBigIntSize%gpd.tee.sockets.version%gpd.tee.sockets.tcp.version%gpd.tee.internalCore.version%lib/l
Index: 1561
%0
Index: 1562
Log entry: %ibutee/tcb.c%total_size >= _tls_size%TCB allocation failed (%zu bytes)%DTV allocation failed (%zu by
Index: 1563
%0
Index: 1564
Log entry: %tes)%dl_phdr_info allocation failed%__utee_tcb_init%dl_iterate_phdr%0123456789ABCDEF%EFE021C2645FD1D
Index: 1565
%0
```

```
Index: 1518
Log entry: %tentObject failed 0x%x%OpenPersistentObject failed 0x%x%New persistent object.%Updating existing per
Index: 1519
%0
Index: 1520
Log entry: %sistent object.%ReadObjectData failed 0x%x%%02u:%02u:%02u.%03u%data in log data [%zu]: %.*s%WriteObj
Index: 1521
%0
Index: 1522
Log entry: %ectData failed 0x%x%get log entry%Variable de prueba pointer: %p%Variable de prueba pointer: %s%bad
Index: 1523
%0
Index: 1524
Log entry: %param types%Persistent object not found%Log pointer: %p%Test: %s%Copying bytes for timestamp%Return
Index: 1525
%0
Index: 1526
Log entry: %address: %p%Stack address: %p%Out of bounds access%Variable de prueba%unsecure storing%unsecure retr
Index: 1527
%0
Index: 1528
Log entry: %ieving%No secret stored%updateLog%G8%get_log_entry%store_secret%retrieve_secret%Parameter type misma
Index: 1529
%0
Index: 1530
Log entry: %tch!%validate password is being called%Password pointer: %x%Got value: %s from NW 1%Length received
Index: 1531
%0
Index: 1532
```

Attacks – Race condition

- Baseline test:
  1. It executes 9 commands serially.
  2. Check whether all command are registered in the log.
- Pushing parallel execution.
  1. It executes 9 commands in parallel.
  2. Check whether all commands are registered in the log.

DEMO

Politecnico di Torino

Thank you!

Questions?